

TypeScript

Table of Contents

classes

decorators

generics

tipos

type-operators

TypeScript Classes

TS

```
class Car {
    engine: string;

    constructor(engine: string) {
        this.engine = engine;
    }

    disp(): void {
        console.log("Function displays Engine is : " + this.engine);
    }
}

let obj = new Car("XXSY1");
console.log("Reading attribute value Engine as : " + obj.engine);
obj.disp();
```

Output:

TEXT

```
Reading attribute value Engine as : XXSY1
Function displays Engine is : XXSY1
```

Forma corta

TS

```
class Heroe {
    static avgAge: number = 35;

    constructor(
        private name: string,
        private team: string,
        private realName?: string
    ) { }
}

const antman: Heroe = new Heroe('Antman', 'Capitan');
console.log(antman);
```

Forma larga

TS

```
class Heroe {
  private name: string;
  private team: string;
  private realName?: string;
  static avgAge: number = 35;

  constructor(name: string, team: string, realName?: string) {
    this.name = name;
    this.team = team;
    this.realName = realName;
  }
}

const antman: Heroe = new Heroe('Antman', 'Capitan');
console.log(antman);
```

Class Inheritance

TS

```
class Shape {
  Area: number;

  constructor(a: number) {
    this.Area = a;
  }
}

class Circle extends Shape {
  disp(): void {
    console.log("Area of the circle: " + this.Area);
  }
}

let obj = new Circle(223);
obj.disp();
```

Output:

TEXT

Area of the Circle: 223

Class Inheritance and Method Overriding

TS

```
class PrinterClass {
    doPrint(): void {
        console.log("doPrint() from Parent called...");
    }
}

class StringPrinter extends PrinterClass {
    doPrint(): void {
        console.log("doPrint() is printing a string...");
    }
}

let obj = new StringPrinter();
obj.doPrint();
```

The static Keyword

TS

```
class StaticMem {
    static num: number;

    static disp(): void {
        console.log("The value of num is " + StaticMem.num);
    }
}

StaticMem.num = 12;
StaticMem.disp();
```

Output:

TEXT

The value of num is 12

The instanceof Operator

TS

```
class Person { }

let obj = new Person();
let isPerson = obj instanceof Person;
console.log("is obj an instance of Person? " + isPerson);
```

Output:

TEXT

is obj an instance of Person? true

Data Hiding

Access Specifiers

Specifier	Description
public	Universal accessibility. Data members in a class are public by default.
private	Accessible only within the class that defines them.
protected	Accessible within the same class and by child classes.

Public

TS

```
class Person {
    name: string;
}

const person = new Person();
person.name = "Jane";
```

Private

TS

```
class Encapsulate {
    str: string = "hello";
    private str2: string = "world";
}

var obj = new Encapsulate();
console.log(obj.str);    // accessible
console.log(obj.str2);  // compilation Error – str2 is private
```

Protected

TS

```
class Person {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    protected greet() {
        return `Hello, my name is ${this.name}`;
    }
}

class Employee extends Person {
    private position: string;

    constructor(name: string, position: string) {
        super(name);
        this.position = position;
    }

    public introduce() {
        return `${this.greet()} and I am a ${this.position}`;
    }
}

const emp = new Employee("Alice", "Developer");
console.log(emp.introduce()); // "Hello, my name is Alice and I am a Developer"

// Accesos inválidos:
// console.log(emp.name);    // Error: 'name' is protected
// console.log(emp.greet()); // Error: 'greet' is protected
```

Getters y Setters

TS

```
class TemperatureConverter {
    private _celsius: number = 0;

    get celsius(): number {
        return this._celsius;
    }

    set celsius(value: number) {
        if (value ≥ -273.15) {
            this._celsius = value;
        } else {
            console.error("Invalid temperature. Must be greater than or e
        }
    }

    get fahrenheit(): number {
        return (this._celsius * 9 / 5) + 32;
    }

    set fahrenheit(value: number) {
        this._celsius = (value - 32) * 5 / 9;
    }
}

const temperatureConverter = new TemperatureConverter();
temperatureConverter.celsius = 25;
console.log("Temperature in Celsius:", temperatureConverter.celsius);
console.log("Temperature in Fahrenheit:", temperatureConverter.fahrenheit);
```

Abstract Classes

Abstract classes are mainly for inheritance where other classes may derive from them. We cannot create an instance of an abstract class.

TS

```
abstract class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    display(): void {
        console.log(this.name);
    }

    abstract find(name: string): Person;
}

class Employee extends Person {
    empCode: number;

    constructor(name: string, code: number) {
        super(name);
        this.empCode = code;
    }

    find(name: string): Person {
        return new Employee(name, 1);
    }
}

let emp: Person = new Employee("James", 100);
emp.display(); // James
let emp2: Person = emp.find('Steve');
```

Constructor en clase abstracta

Las clases abstractas pueden tener constructor. Aunque no puedes instanciarlas directamente, las subclases llaman al constructor mediante `super()` para inicializar propiedades o configuración común.

```
abstract class Animal {
    constructor(public name: string) {
        console.log(`Animal ${name} is created.`);
    }

    abstract makeSound(): void;
}

class Dog extends Animal {
    constructor(name: string) {
        super(name);
    }

    makeSound(): void {
        console.log(`${this.name} says: Woof!`);
    }
}

class Cat extends Animal {
    constructor(name: string) {
        super(name);
    }

    makeSound(): void {
        console.log(`${this.name} says: Meow!`);
    }
}

const dog = new Dog("Buddy");    // Animal Buddy is created.
dog.makeSound();                // Buddy says: Woof!
const cat = new Cat("Whiskers"); // Animal Whiskers is created.
cat.makeSound();                // Whiskers says: Meow!
```

Variables abstractas

```
abstract class Animal {
  abstract species: string;

  constructor(public name: string) {}

  describe(): void {
    console.log(`${this.name} is a ${this.species}.`);
  }

  abstract makeSound(): void;
}

class Dog extends Animal {
  species: string = "Dog";

  makeSound(): void {
    console.log(`${this.name} says: Woof!`);
  }
}

class Cat extends Animal {
  species: string = "Cat";

  makeSound(): void {
    console.log(`${this.name} says: Meow!`);
  }
}

const dog = new Dog("Buddy");
dog.describe(); // Buddy is a Dog.
dog.makeSound(); // Buddy says: Woof!

const cat = new Cat("Whiskers");
cat.describe(); // Whiskers is a Cat.
cat.makeSound(); // Whiskers says: Meow!
```

Private Constructors (Singleton Pattern)

```
class Logger {
    private static instance: Logger;

    private constructor() {
        console.log("Logger initialized.");
    }

    public static getInstance(): Logger {
        if (!Logger.instance) {
            Logger.instance = new Logger();
        }
        return Logger.instance;
    }

    public log(message: string): void {
        console.log(`Log entry: ${message}`);
    }
}

const logger1 = Logger.getInstance();
logger1.log("First log entry."); // Log entry: First log entry.

const logger2 = Logger.getInstance();
logger2.log("Second log entry."); // Log entry: Second log entry.

console.log(logger1 === logger2); // true
```

Decorators

Los decorators en TypeScript son funciones que permiten modificar o añadir comportamiento a clases, métodos, propiedades o parámetros, sin modificar directamente su código.

Tipos:

- Class Decorators
- Method Decorators
- Accessor Decorators (getters/setters)
- Property Decorators
- Parameter Decorators

Configuración

Se activan en `tsconfig.json` :

JSON

```
{
  "compilerOptions": {
    "target": "ES5",
    "experimentalDecorators": true
  }
}
```

Sintaxis

Se utilizan precediendo el elemento a decorar con `@decoratorName` :

TS

```
@classDecorator
class Person {
  @propertyDecorator
  public name: string;

  @accessorDecorator
  get fullName() {
    // ...
  }

  @methodDecorator
  printName(@parameterDecorator prefix: string) {
    // ...
  }
}
```

Múltiples decorators

TS

```
@decoratorA
@decoratorB
class Person { }
```

Decorator de Clase

Recibe el constructor de la clase como argumento. Puede modificar el constructor o agregar nuevas propiedades o métodos.

Parámetros:

- `target: Function` — El constructor de la clase decorada.

TS

```
function Logger(target: Function) {
  console.log(`Clase registrada: ${target.name}`);
}

@Logger
class Person {
  name = "Alan";
}
```

TS

```
function Logger(constructor: Function) {
    console.log("Clase inicializada:", constructor.name);
}

@Logger
class Persona {
    constructor(public nombre: string) {}
}

const persona = new Persona("Juan");
// Imprime: "Clase inicializada: Persona"
```

Modificar el comportamiento

TS

```
function replaceConstructor(constructor: Function) {
    return class {
        newProperty = "Soy una nueva propiedad";
        hello = "Hola desde el decorador";

        constructor() {
            console.log("Constructor modificado");
        }
    };
}

@replaceConstructor
class MyClass {
    constructor() {
        console.log("Constructor original de MyClass");
    }
}

const instance = new MyClass() as any;
console.log(instance.hello); // Hola desde el decorador
console.log(instance.newProperty); // Soy una nueva propiedad
```

Decorador de Propiedad

Recibe dos parámetros:

- `target: any` — El prototipo de la clase (instancia) o el constructor (estática).
- `key: string` — El nombre de la propiedad decorada.

TS

```
function ReadOnly(target: any, key: string) {
    Object.defineProperty(target, key, {
        writable: false,
        configurable: false,
    });
}

class User {
    @ReadOnly
    role = "Admin";
}

const user = new User();
(user as any).role = "User";
console.log(user.role); // "Admin"
```

TS

```
function logAccess(target: any, key: string) {
    let value = target[key];

    const getter = () => {
        console.log(`Accediendo a la propiedad "${key}"`);
        return value;
    };

    Object.defineProperty(target, key, {
        get: getter,
        set: (newValue) => { value = newValue; },
        enumerable: true,
        configurable: true,
    });
}

class MyClass2 {
    @logAccess
    myProperty: string = "Hola";
}

const instance2 = new MyClass2();
console.log(instance2.myProperty);
// Accediendo a la propiedad "myProperty"
```

Decorador de Método

Recibe tres parámetros:

- `target: any` — Prototipo de la clase (instancia) o constructor (estática).
- `key: string` — Nombre del método decorado.
- `descriptor: PropertyDescriptor` — Permite modificar el comportamiento del método.

TS

```
function modifyReturnValue(target: any, key: string, descriptor: PropertyDescriptor) {
    const originalMethod = descriptor.value;

    descriptor.value = function (...args: any[]) {
        const result = originalMethod.apply(this, args);
        return `Modificado: ${result}`;
    };
}

class MyClass {
    @modifyReturnValue
    greet(name: string) {
        return `Hola, ${name}`;
    }
}

const instance = new MyClass();
console.log(instance.greet("Juan")); // Modificado: Hola, Juan
```

TS

```
function LogMethod(target: any, key: string, descriptor: PropertyDescriptor) {
    const originalMethod = descriptor.value;

    descriptor.value = function (...args: any[]) {
        console.log(`Llamando ${key} con argumentos:`, args);
        return originalMethod.apply(this, args);
    };
}

class Calculator {
    @LogMethod
    add(a: number, b: number) {
        return a + b;
    }
}

const calc = new Calculator();
console.log(calc.add(5, 3)); // Llamando add con argumentos: [5,3]
```

Decorador de Parámetros

Permiten modificar o acceder a información sobre los parámetros de los métodos. No pueden cambiar directamente el valor, pero son útiles para validación, seguimiento o transformación.

Parámetros:

- `target: any` — Prototipo de la clase.
- `key: string | symbol` — Nombre del método.
- `parameterIndex: number` — Índice del parámetro en la función.

TS

```
function LogParam(target: any, key: string, parameterIndex: number) {
    console.log(`Parámetro decorado en ${key}, índice: ${parameterIndex}`)
}

class Person {
    greet(@LogParam message: string) {
        console.log(message);
    }
}

const p = new Person();
p.greet("Hola!"); // Parámetro decorado en greet, índice: 0
```

TS

```
function positiveNumber(target: any, propertyKey: string, parameterIndex: number): void {
    const originalMethod = target[propertyKey];

    target[propertyKey] = function (...args: any[]) {
        if (args[parameterIndex] ≤ 0) {
            throw new Error(`El parámetro en la posición ${parameterIndex} no es positivo`);
        }
        return originalMethod.apply(this, args);
    };
}

class MyClass {
    greet(message: string, @positiveNumber count: number) {
        return `${message} se ha repetido ${count} veces`;
    }
}

const instance = new MyClass();
console.log(instance.greet("Hola", 3)); // Hola se ha repetido 3 veces
console.log(instance.greet("Hola", -1)); // Error
```

Decorador de Accesorio (Getter / Setter)

Parámetros:

- `target: any` — Prototipo de la clase.
- `key: string | symbol` — Nombre del accesorio.
- `descriptor: PropertyDescriptor` — Modifica el comportamiento del accesorio.

TS

```
function Capitalize(target: any, key: string, descriptor: PropertyDescriptor) {
    const originalGetter = descriptor.get;

    descriptor.get = function () {
        const result = originalGetter?.apply(this);
        return result.toUpperCase();
    };
}

class Person {
    private _name = "alan";

    @Capitalize
    get name() {
        return this._name;
    }
}

const p = new Person();
console.log(p.name); // ALAN
```

Generics

Los genéricos son una característica que permite crear componentes o funciones reutilizables que pueden trabajar con diferentes tipos de datos, sin perder el control sobre los tipos.

¿Por qué usar Genéricos?

Sin genéricos, usando `any` se pierde la verificación de tipos:

TS

```
function identity(arg: any): any {  
    return arg;  
}
```

Con genéricos, el tipo se conserva:

TS

```
function identity<T>(arg: T): T {  
    return arg;  
}
```

Genéricos en Clases

TS

```
class Caja<T> {
  contenido: T;

  constructor(contenido: T) {
    this.contenido = contenido;
  }

  obtenerContenido(): T {
    return this.contenido;
  }
}

const cajaDeNumero = new Caja<number>(123);
console.log(cajaDeNumero.obtenerContenido()); // 123

const cajaDeString = new Caja<string>("Hola");
console.log(cajaDeString.obtenerContenido()); // Hola
```

Genéricos con Funciones

TS

```
function mostrarElementos<T>(items: T[]): void {
  items.forEach(item => console.log(item));
}

mostrarElementos<number>([1, 2, 3]);
mostrarElementos<string>(["uno", "dos", "tres"]);
```

Ejemplo con interfaces

TS

```
interface ProductoElectronico {
    id: number;
    nombre: string;
    potencia: number;
}

interface ProductoRopa {
    id: number;
    nombre: string;
    talla: string;
}

class Inventario<T> {
    private items: T[] = [];

    agregar(item: T): void {
        this.items.push(item);
    }

    obtenerTodos(): T[] {
        return this.items;
    }
}

const inventarioElectronico = new Inventario<ProductoElectronico>();
inventarioElectronico.agregar({ id: 1, nombre: 'Televisor', potencia: 150 });
console.log(inventarioElectronico.obtenerTodos());
// [{ id: 1, nombre: 'Televisor', potencia: 150 }]

const inventarioRopa = new Inventario<ProductoRopa>();
inventarioRopa.agregar({ id: 2, nombre: 'Camiseta', talla: 'M' });
console.log(inventarioRopa.obtenerTodos());
// [{ id: 2, nombre: 'Camiseta', talla: 'M' }]
```

Constraints (T extends)

TS

```

class Animal {
  nombre: string;
  constructor(nombre: string) {
    this.nombre = nombre;
  }
  hacerRuido(): void {
    console.log(`${this.nombre} está haciendo ruido.`);
  }
}

class Perro extends Animal {
  ladrar(): void {
    console.log(`${this.nombre} está ladrando.`);
  }
}

class Gato extends Animal {
  maullar(): void {
    console.log(`${this.nombre} está maullando.`);
  }
}

function entrenarAnimal<T extends Animal>(animal: T): void {
  animal.hacerRuido();
}

const miPerro = new Perro("Firulais");
entrenarAnimal(miPerro); // Firulais está haciendo ruido.

const miGato = new Gato("Michi");
entrenarAnimal(miGato); // Michi está haciendo ruido.

```

`T extends Animal` — la función solo acepta instancias de `Animal` o cualquier clase que extienda `Animal`.

Record

TS

```

function getProperty(obj: Record<string | number, any>, property: string)
  return obj[property];
}

getProperty({ test: 'testValue' }, 'test'); // 'testValue'
getProperty({ 2: [1, 2, 3] }, 2); // [1, 2, 3]
getProperty([ { a: 0 }, { a: 1 }, { a: 2 } ], 2); // { a: 2 }

```

Interfaz Genérica + Clase

TS

```
interface QueueInterface<T> {
  push(value: T): void;
  pop(): T | undefined;
  getValue(): T[];
}

class Queue<T> implements QueueInterface<T> {
  queue: T[];

  constructor() {
    this.queue = [];
  }

  push(value: T): void {
    this.queue.push(value);
  }

  pop(): T | undefined {
    const element = this.queue.shift();
    console.log(element);
    return element;
  }

  getValue(): T[] {
    console.log(this.queue);
    return this.queue;
  }
}

const numberQueue = new Queue<number>();
numberQueue.push(1);
numberQueue.push(2);
numberQueue.push(3);
numberQueue.pop(); // 1
numberQueue.getValue(); // [2, 3]

const objectQueue = new Queue<{}>();
objectQueue.push({ first: 1 });
objectQueue.push({ second: 2 });
objectQueue.pop(); // { first: 1 }
objectQueue.getValue(); // [{ second: 2 }]
```

TypeScript Fundamentals

TypeScript is a programming language developed and maintained by Microsoft. It is a superset of JavaScript that adds static typing and other features.

- Type annotations (string, number, boolean, etc.)
- Interfaces, generics, enums, utility types
- Compile-time type checking
- Modern language features before they are natively available in browsers

It compiles down to JavaScript — TypeScript code is not directly executed by browsers. The TypeScript compiler (tsc) converts it into plain JavaScript.

TypeScript Commands

Command	Description
<code>npm install -g typescript</code>	Install TypeScript globally
<code>npm install typescript --save-dev</code>	Install TypeScript locally
<code>npx tsc --version</code>	Check version
<code>tsc example.ts</code>	Compile a single file
<code>tsc</code>	Compile all files (with tsconfig.json)
<code>tsc --init</code>	Generate tsconfig.json
<code>tsc --watch</code>	Watch mode
<code>control + c</code>	Exit watch mode

tsconfig.json Configurations

JSON

```
"noImplicitAny": true // Ensures type safety
"sourceMap": true // Debug TypeScript in browser
"removeComments": true // Remove comments in compiled output
```

Exclude files/folders:

JSON

```
"exclude": ["node_modules"]
```

Include files/folders:

JSON

```
"include": ["src"]
```

Output file (AMD):

JSON

```
"module": "amd",  
"outFile": "./main.js"
```

Hoy en día se prefiere usar un empaquetador como Webpack, Vite o Rollup.

Decorators

Se activan en tsconfig.json:

JSON

```
"experimentalDecorators": true
```

Data Types

We assign data types by placing a colon after the variable name:

TEXT

```
const variableName: variableType = value
```

Simple Types

TS

```
const myBool: boolean = false;  
let myString: string = 'bacon';  
let age: number = 30;  
let price: number = 19.99;  
let negative: number = -20;  
const bigNumber: bigint = 12345678901234567890n;
```

Symbol

TS

```
let symbol1 = Symbol();
console.log(symbol1);

let symbol2 = Symbol("abc");
console.log(symbol2);
```

Special Types

Type: any — Disables type checking.

TS

```
let v: any = true;
v = "string"; // no error
Math.round(v); // no error as it can be "any" type
```

NOTE: When you don't specify a type and TypeScript can't infer it from context, the compiler will typically default to any.

Type: unknown — Requires type verification before use.

TS

```
let value: unknown;
value = "Hola";
value = 42;
value = true;

// Type verification required:
if (typeof value === "string") {
    console.log(value.toUpperCase());
}

if (typeof value === "number") {
    console.log(value.toFixed(2));
}
```

Without type verification, TypeScript will throw an error:

TS

```
let value: unknown;
value = "Hola";
console.log(value.toUpperCase()); // Error
```

Type: undefined

TS

```
let x: number | undefined;  
console.log(x); // undefined
```

Type: null

TS

```
let y: number | null = null;  
console.log(y); // null
```

Arrays

TS

```
let numbers: number[] = [1, 2, 3, 4, 5];  
let strings: string[] = ["hello", "world"];  
  
// Multidimensional  
let matrix: number[][] = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];  
console.log(matrix[0][0]); // 1  
  
// Mixed arrays  
let mixedArray: (number | string | boolean)[] = [1, "hello", true, 2];  
let anotherMixedArray: Array<number | string | boolean> = [2, "world", false];
```

Tuples

TS

```
let person: [number, string] = [1, "John Doe"];
```

Enum

Define a set of named values.

Numeric enums:

TS

```
enum CardinalDirections {
    North,
    East,
    South,
    West
}

console.log(CardinalDirections.North); // 0
console.log(CardinalDirections.West); // 3

enum Direction {
    Up,      // 0
    Down,    // 1
    Left,    // 2
    Right    // 3
}

let direction: Direction = Direction.Up;
console.log(direction); // 0
```

String enums:

TS

```
enum Direction {
    Up = "UP",
    Down = "DOWN",
    Left = "LEFT",
    Right = "RIGHT"
}

let direction: Direction = Direction.Left;
console.log(direction); // "LEFT"
```

Objects

TS

```
let product: {  
  name: string;  
  price: number;  
};  
product = {  
  name: "TV",  
  price: 300  
};  
  
const car: { type: string; model: string; year: number } = {  
  type: "Toyota",  
  model: "Corolla",  
  year: 2009  
};
```

Union Operator (|)

Combine multiple possible types.

TS

```
let value: string | number | boolean | null | undefined;  
value = "texto";  
value = 42;  
value = true;  
value = null;  
value = undefined;  
  
type WindowStates = "open" | "closed" | "minimized";  
type LockStates = "locked" | "unlocked";  
type PositiveOddNumbersUnderTen = 1 | 3 | 5 | 7 | 9;
```

Functions

TS

```
// Function declaration
function getTime(): number {
    return new Date().getTime();
}

// Function expression
const multiply = function (a: number, b: number): number {
    return a * b;
};

// Arrow function
const divide = (a: number, b: number): number => {
    return a / b;
};

// Void
function printHello(): void {
    console.log('Hello!');
}
```

Optional Parameters:

TS

```
function greet(firstName: string, lastName?: string): string {
    if (lastName) {
        return `Hello, ${firstName} ${lastName}`;
    }
    return `Hello, ${firstName}`;
}
console.log(greet("Alice"));           // "Hello, Alice"
console.log(greet("Alice", "Johnson")); // "Hello, Alice Johnson"
```

Default Parameters:

TS

```
function subtract(a: number, b: number = 5): number {
    return a - b;
}
console.log(subtract(10)); // 5
console.log(subtract(10, 3)); // 7
```

Rest Parameters:

TS

```
function concatenate(separator: string, ...strings: string[]): string {  
    return strings.join(separator);  
}  
console.log(concatenate(", ", "Alice", "Bob", "Charlie")); // "Alice, Bob, Charlie"
```

Parameters Omission:

TS

```
const myFunction = (_, secondParam) => {  
    console.log(secondParam);  
};  
myFunction("No lo uso", "Este sí se imprime"); // Output: "Este sí se imprime"  
  
const numbers = [10, 20, 30];  
const indexedNumbers = numbers.map( (_, index) => index);  
console.log(indexedNumbers); // Output: [0, 1, 2]
```

Type function:

TS

```
const addNumbers = (a: number, b: number) => a + b;  
const sayHello = (name: string) => `Hola ${name}`;  
  
let myFunction: Function;  
myFunction = addNumbers;  
console.log(myFunction(1, 2));  
myFunction = sayHello;  
console.log(myFunction('Alan'));  
  
let myFunction2: (y: number, z: number) => number;  
myFunction2 = addNumbers;  
console.log(myFunction2(1, 2));  
  
let myFunction3: (y: string) => string;  
myFunction3 = sayHello;  
console.log(myFunction3("Eduardo"));
```

Objects with methods:

TS

```
let flash: { name: string; age?: number; powers: string[]; getName?: () => string } = {
  name: 'Barry Allen',
  age: 24,
  powers: ['Súper velocidad', 'Viajar en el tiempo']
};

flash = {
  name: 'Clark Kent',
  powers: ['Súper fuerza'],
  getName() {
    return this.name;
  }
};

if (flash.getName) {
  console.log(flash.getName());
}
```

Multiple types:

TS

```
type Person = {
  name: string;
  age?: number;
  getName?: () => string;
};

let myCustomVariable: string | number | Person = "Alan";
console.log(typeof myCustomVariable);
myCustomVariable = 20;
console.log(typeof myCustomVariable);
myCustomVariable = {
  name: "John",
  age: 22,
};
console.log(typeof myCustomVariable);
```

Type Aliases

TS

```
type CarYear = number;
type CarType = string;
type CarModel = string;
type Car = {
  year: CarYear;
  type: CarType;
  model: CarModel;
};
const carYear: CarYear = 2001;
const carType: CarType = "Toyota";
const carModel: CarModel = "Corolla";
const car: Car = {
  year: carYear,
  type: carType,
  model: carModel,
};
```

Extender un type alias

TS

```
type User = {
  name: string;
  age: number;
};

type ExtendedUser = User & {
  email: string;
};

const user: ExtendedUser = {
  name: "Alan",
  age: 30,
  email: "alan@example.com",
};
```

Quitar propiedades (Omit)

TS

```
type User = {
  name: string;
  age: number;
  email: string;
};

type UserWithoutEmail = Omit<User, "email" | "age">;
const user: UserWithoutEmail = {
  name: "Alan",
};
```

Pick

TS

```
type User = {
  name: string;
  age: number;
  email: string;
};

type UserPreview = Pick<User, "name" | "email">;
const user: UserPreview = {
  name: "Alan",
  email: "alan@example.com",
};
```

Lookup Types

TS

```
type User = {
  name: string;
  age: number;
  email: string;
};

type UserName = User["name"]; // string
const userName: UserName = "Alan";
```

Partial (hacer propiedades opcionales)

TS

```
type User = {
  name: string;
  age: number;
  email: string;
};

type UpdateUser = Partial<User>;
const userUpdate: UpdateUser = {
  name: "Jane Doe",
};
```

Partial + Omit

TS

```
type UpdateUser = Partial<Omit<User, "email">>;
// or
type UpdateUser = Omit<User, "email"> & Partial<User>;

const userUpdate: UpdateUser = {
  name: "John Doe",
  age: 30,
};
```

Partial + Omit + nuevas propiedades

TS

```
type UpdateUser = Partial<Omit<User, 'email'>> & {
  isActive?: boolean;
};

const userUpdate: UpdateUser = {
  name: "John Doe",
  age: 30,
  isActive: true,
};
```

Wildcards / Mapped Types

TS

```
export type User = {  
  id: number;  
  name: Capitalize<string>;  
  login: Lowercase<string>;  
};
```

Interfaces

Interfaces are similar to type aliases, except they only apply to object types.

Simple

TS

```
interface Rectangle {  
  height: number;  
  width: number;  
}  
  
const rectangle: Rectangle = {  
  height: 20,  
  width: 10,  
};
```

Compleja

TS

```
interface Client {
  name: string;
  age?: number;
  address: Address;
}

interface Address {
  id: number;
  zip: number;
  city: string;
}

const cliente: Client = {
  name: "Alan",
  age: 20,
  address: {
    id: 1,
    zip: 12345,
    city: "GDL",
  },
};
```

Con clases

TS

```
interface Animal {
  nombre: string;
  hacerSonido(): void;
}

class Perro implements Animal {
  nombre: string;
  constructor(nombre: string) {
    this.nombre = nombre;
  }
  hacerSonido() {
    console.log("Guau guau");
  }
}

const miPerro = new Perro("Rex");
miPerro.hacerSonido();
```

Implementando varias interfaces

```
interface GestorArchivos {
  abrirArchivo(ruta: string): void;
  guardarArchivo(ruta: string, contenido: string): void;
}

interface RegistroCambios {
  registrarCambio(cambio: string): void;
  mostrarRegistro(): void;
}

class Documento implements GestorArchivos, RegistroCambios {
  private registro: string[] = [];

  abrirArchivo(ruta: string): void {
    console.log(`Archivo abierto desde: ${ruta}`);
  }

  guardarArchivo(ruta: string, contenido: string): void {
    console.log(`Archivo guardado en: ${ruta}`);
    this.registrarCambio(`Guardado en ${ruta}`);
  }

  registrarCambio(cambio: string): void {
    this.registro.push(cambio);
  }

  mostrarRegistro(): void {
    console.log("Registro de cambios:");
    this.registro.forEach((cambio, index) => {
      console.log(`${index + 1}: ${cambio}`);
    });
  }
}

const doc = new Documento();
doc.abrirArchivo("/path/to/file.txt");
doc.guardarArchivo("/path/to/file.txt", "Nuevo contenido");
doc.mostrarRegistro();
```

Interfaces en las funciones

TS

```
interface OperacionMatematica {  
    (a: number, b: number): number;  
}  
  
let addNumbersFunction: OperacionMatematica;  
addNumbersFunction = (a: number, b: number) => {  
    return a + b;  
};
```

Herencia en las interfaces

TS

```
interface Persona {  
    nombre: string;  
    edad: number;  
}  
  
interface Direccion {  
    calle: string;  
    ciudad: string;  
}  
  
interface Contacto extends Persona, Direccion {  
    telefono: string;  
}  
  
const contacto: Contacto = {  
    nombre: "Juan",  
    edad: 30,  
    calle: "Calle Falsa 123",  
    ciudad: "Springfield",  
    telefono: "555-1234",  
};  
console.log(contacto);
```

Declaration Merging

TS

```
interface User {
  name: string;
  age: number;
}

// Declaration merging – adds new properties to existing interface
interface User {
  email: string;
}

const user: User = {
  name: "Alan",
  age: 30,
  email: "alan@example.com",
};
```

ReadOnly

TS

```
class Employee {
  readonly empCode: number;
  empName: string;
  constructor(code: number, name: string) {
    this.empCode = code;
    this.empName = name;
  }
}

let emp = new Employee(10, "John");
// emp.empCode = 20; // Compiler Error

interface IEmployee {
  readonly empCode: number;
  empName: string;
}

let empObj: IEmployee = {
  empCode: 1,
  empName: "Steve",
};
// empObj.empCode = 100; // Compiler Error
```

Quitar atributos de una interfaz (Omit)

TS

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  password: string;  
}  
  
type PublicUser = Omit<User, "password">;  
  
const user: PublicUser = {  
  id: 1,  
  name: "Alan",  
  email: "alan@example.com",  
};
```

Namespaces

A namespace is a way to organize code into logical groups and avoid naming collisions.

TS

```
namespace Validations {  
  export const validateText = (text: string): boolean => {  
    return text.length > 3;  
  };  
  
  export const validateDate = (myDate: Date): boolean => {  
    return !isNaN(myDate.valueOf());  
  };  
}  
  
console.log(Validations.validateText('hola'));  
const fechaValida = new Date("2024-09-13");  
console.log(Validations.validateDate(fechaValida));
```

Not Null Assertion Operator (!)

TS

```
document.getElementById('root')!
```

Type Operators

`typeof` (type context)

Captura el tipo de una variable o expresión en tiempo de compilación.

TS

```
const name = "Alan";
type NameType = typeof name; // string

const user = { name: "Alan", age: 30 };
type UserType = typeof user;
// { name: string; age: number; }

function greet(name: string): string {
    return `Hello, ${name}`;
}
type GreetType = typeof greet;
// (name: string) => string
```

`keyof`

Obtiene una unión de las claves de un tipo.

TS

```
type Person = { name: string; age: number; email: string };
type PersonKeys = keyof Person; // "name" | "age" | "email"

function getValue<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const person: Person = { name: "Alan", age: 30, email: "alan@example.com" };
getValue(person, "name"); // string
getValue(person, "age"); // number
// getValue(person, "invalid"); // Error
```

Indexed Access Types (`T[K]`)

Accede al tipo de una propiedad específica.

TS

```
type Person = { name: string; age: number; email: string };

type NameType = Person["name"]; // string
type AgeType = Person["age"];   // number
type NameOrAge = Person["name" | "age"]; // string | number

type AllValues = Person[keyof Person]; // string | number

const users = [
  { name: "Alice", age: 25 },
  { name: "Bob", age: 30 },
];
type UserFromArray = (typeof users)[number];
// { name: string; age: number; }
```

Conditional Types (`T extends U ? X : Y`)

Tipos que dependen de una condición evaluada en tiempo de compilación.

TS

```
type IsString<T> = T extends string ? "yes" : "no";

type A = IsString<string>; // "yes"
type B = IsString<number>; // "no"

// Distributive conditional types
type ToArray<T> = T extends unknown ? T[] : never;
type Result = ToArray<string | number>; // string[] | number[]

// Con constraint
type ExtractStrings<T> = T extends string ? T : never;
type StringsOnly = ExtractStrings<string | number | boolean>; // string
```

Built-in Conditional Types

TS

```
type T1 = Exclude<string | number | boolean, boolean>; // string | number
type T2 = Extract<string | number | boolean, boolean>; // boolean
type T3 = NonNullable<string | number | null | undefined>; // string | number
type T4 = ReturnType<() => string>; // string
type T5 = InstanceType<typeof Date>; // Date
```

Mapped Types (`{ [K in T]: ... }`)

Transforman las propiedades de un tipo existente.

TS

```
type Readonly<T> = {
  readonly [K in keyof T]: T[K];
};

type Partial<T> = {
  [K in keyof T]?: T[K];
};

type Nullable<T> = {
  [K in keyof T]: T[K] | null;
};

type Person = { name: string; age: number };
type ReadonlyPerson = Readonly<Person>;
// { readonly name: string; readonly age: number; }

// Renombrar propiedades con as
type Getters<T> = {
  [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K];
};

type PersonGetters = Getters<{ name: string; age: number }>;
// { getName: () => string; getAge: () => number; }
```

Built-in Mapped Types

TS

```
type P1 = Partial<{ name: string; age: number }>;
type P2 = Required<{ name?: string; age?: number }>;
type P3 = Readonly<{ name: string; age: number }>;
type P4 = Pick<{ name: string; age: number; email: string }, "name" | "email">;
type P5 = Record<"a" | "b", number>; // { a: number; b: number; }
```

Template Literal Types

Combinan string literals con union types para crear nuevos strings en el sistema de tipos.

TS

```
type EventName = "click" | "focus" | "blur";
type HandlerName = `on${Capitalize<EventName>}`;
// "onClick" | "onFocus" | "onBlur"

type Vertical = "top" | "bottom";
type Horizontal = "left" | "right";
type Position = `${Vertical}-${Horizontal}`;
// "top-left" | "top-right" | "bottom-left" | "bottom-right"

type Size = "sm" | "md" | "lg";
type Color = "red" | "blue";
type ClassName = `${Size}-${Color}`;
// "sm-red" | "sm-blue" | "md-red" | "md-blue" | "lg-red" | "lg-blue"
```

Intrinsic String Types

TS

```
type A = Uppercase<"hello">; // "HELLO"
type B = Lowercase<"HELLO">; // "hello"
type C = Capitalize<"hello">; // "Hello"
type D = Uncapitalize<"Hello">; // "hello"
```

infer

Extrae tipos dentro de conditional types.

TS

```
type ReturnType<T> = T extends (...args: unknown[]) => infer R ? R : never;

type Fn = (a: number, b: string) => boolean;
type FnReturn = ReturnType<Fn>; // boolean

type ElementType<T> = T extends (infer U)[] ? U : never;
type ArrElement = ElementType<string[]>; // string

type PromiseValue<T> = T extends Promise<infer V> ? V : never;
type Value = PromiseValue<Promise<number>>; // number
```